EXTENDED ABSTRACT
ACTIVITY TRACKING WITH THE REACHY MINI ROBOT

Artificial Intelligence
Lectoraat Data Intelligence
Zuyd Hogeschool

Caio de Alcantara Santos (1234567890)
Cas Limpens (1234567890)
Donné Deguelle (1234567890)
Gustavo Gonçalves da Costa (1234567890)
Kanak Manjari (supervisor)
Maarten Vaessen (supervisor)

2026/2027, Q1
01-01-2026

Abstract

Home-based exercise monitoring could reduce repeated hospital visits, but it requires reliable activity recognition in unconstrained environments. This project investigates whether the Reachy Mini robot can provide person detection and activity monitoring while giving feedback as an exercise buddy. A feasibility prototype was developed using a camera-to-pose-to-classifier pipeline. The system tracks a person, extracts COCO-17 keypoints, computes temporal features, and classifies activities with random forest inference. The shared pipeline can run both in simulation and on the physical robot. MoveNet int8 was selected for the robot because of its hardware constraints, and recognized actions trigger antenna responses. Evaluation used subject-separated public skeleton datasets, external datasets, recorded robot footage, and both full-body and waist-up framing. In real-time evaluation, wave, clapping, and jumping jacks were sufficiently reliable for demonstration. Overall, the results support the Reachy Mini as a feasible CPU-based platform for activity-monitoring research, while showing that this prototype is not a CMAS scorer or a clinically validated decision-support system.

1 Introduction

1.1 Background

The Research Centre for Data Intelligence conducts applied research into, among other themes, smart assistants and human-computer interaction. This project was carried out in the context of the centre's "Measuring at Home with AI" research direction. The long-term motivation for that research is to make exercise monitoring more accessible to children with juvenile dermatomyositis (JDM). These children currently perform exercises related to the Childhood Myositis Assessment Scale (CMAS) under the supervision of specialists at the Wilhelmina Children's Hospital (WKZ). Repeated travel to the hospital is burdensome for children and their families, which creates an opportunity for technologies that can support monitoring at home. Automated home monitoring must do more than record a video. It must detect a person, estimate body movement from visual input, distinguish between activities, and communicate its interpretation clearly to the user. It must also operate under practical conditions, such as changing distances, incomplete views of the body, and limited computing resources. The stakeholders for this feasibility study are the Research Centre for Data Intelligence, the project client and researcher, and the children and families who represent the future users of a home-monitoring system. The project requirements were consolidated in a software requirements specification covering perception, robot interaction, activity recognition, feedback, session management, privacy, accuracy, reliability, usability, and safety. The current project addresses the technical feasibility of such a system; it does not develop or validate a medical assessment instrument.

1.2 Goal

The goal of this project is to investigate whether the Reachy Mini can perform person detection and activity monitoring, including expressive feedback, under the computational constraints of the wireless robot. The resulting artefact is a working prototype that runs in simulation and on the physical robot. It contains a shared camera-to-pose-to-classifier pipeline, temporal feature extraction, activity-recognition models, and robot applications that translate recognized actions into antenna responses. Candidate vision models were benchmarked under the robot's CPU constraints, leading to the selection of MoveNet int8 for on-robot pose estimation. Model-training and export tools were also developed so that the robot can perform inference with NumPy-only model files rather than requiring the full training environment. Recognition models were evaluated on public skeleton datasets with subject or footage-group separation, as well as on robot recordings and an external dataset. Users can also record personalized movements and create their own datasets for future recognition models. The prototype recognizes six classes: none, wave, pushup, squat, clapping, and jumping jacks. Evaluation focuses on technical feasibility using simulation, public skeleton datasets, recorded robot footage, an external test dataset, and different camera framings. The robot's camera should ideally capture the whole person, because waist-up framing removes information that is important for some activities. The prototype is not a CMAS scorer, is not clinically validated, and should not be used as a clinical decision-support system. The central research question is:

> To what extent can the Reachy Mini detect a person, recognize physiotherapy-related exercises, and provide expressive feedback while operating within its computational constraints?

The remainder of this report first discusses the theoretical foundations and related solution space. It then describes the prototype and its evaluation method, presents the results, discusses their implications and limitations, and concludes with directions for future development.

2 Methods

This study employed a custom, iterative methodology to investigate and develop an activity-monitoring system for the Reachy Mini. The approach shares similarities with the Design Science Research (DSR) framework, which is suitable for design-oriented research involving the development and evaluation of an artefact [1]. The approach was selected because the technical feasibility of the system was initially uncertain and several technical choices required investigation before implementation.

The methodology consisted of five phases:

1. Research: identifying stakeholder needs, establishing requirements, and investigating existing techniques.
2. Design: selecting suitable models and defining the technical architecture.
3. Realisation: implementing the activity-recognition pipeline and integrating it with the robot.
4. Testing: evaluating model performance and system behaviour under different conditions.

5. Evaluation: assessing feasibility, identifying limitations, and formulating recommendations.

The phases were not strictly sequential. Research informed design decisions, while implementation and testing were conducted iteratively. Results from experiments were used to refine technical choices and improve the artefact. This approach supported early validation and incremental improvement, which suited the exploratory nature of the project [2]. A traditional waterfall approach was considered less suitable because the technical choices were not fully determined at the start and the project required opportunities to incorporate findings during implementation [3].

Figure 1 illustrates the iterative prototyping approach. Figure 2 shows the activities conducted during the research phase.

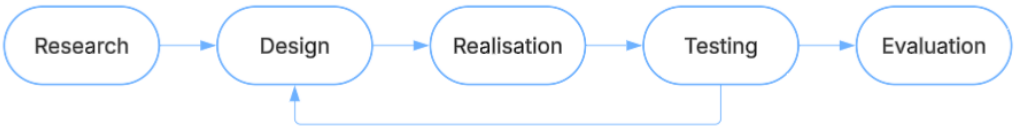


Figure 1: Iterative prototyping approach used in the project.

2.1 Requirements and software requirements specification

The research began with a stakeholder analysis to identify the parties involved and their interests and needs. Functional and non-functional requirements were derived from the project description and meetings with the client. These requirements were consolidated in a software requirements specification (SRS), which served as the reference for design and evaluation.

The functional requirements cover perception and local processing, robot behaviour and interaction, activity and exercise recognition, assessment and feedback, and session management. The non-functional requirements address privacy, recognition accuracy, reliability, usability, and safety. The complete SRS contains the detailed requirement statements and is included as an attachment to the article.

The research was organised around two complementary activities. First, existing techniques for person detection, pose estimation, and activity recognition were investigated. Second, relevant scientific literature, datasets, standards, and guidelines were studied to identify suitable approaches and relevant constraints. The findings were compared against the system requirements and the Reachy Mini's camera and computational capabilities. This process informed the selection of candidate models and the subsequent evaluation of their suitability.

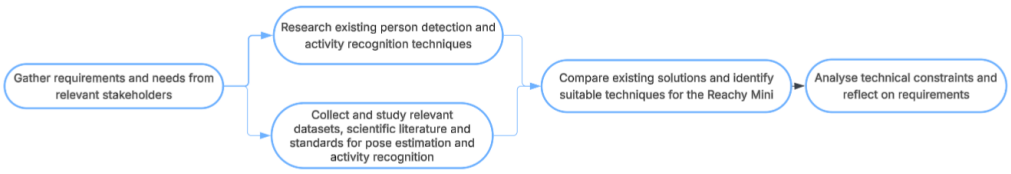


Figure 2: Research activities used to establish requirements and identify suitable techniques for the Reachy Mini.

2.2 Pose-model research and selection

Candidate models for human pose detection and temporal activity analysis were investigated under the Reachy Mini's CPU and camera constraints. The candidates included BlazePose for fast 2D pose detection, ViTPose as a transformer-based 2D estimator, WHAM and CameraHMR for 3D motion or pose-and-shape estimation, MoveNet for lightweight 2D pose estimation, SAM2 for segmentation, PySlowFast for video classification, FSPose for fused-heatmap pose estimation, GraphEnet for graph-based pose estimation, YOLOPose for pose estimation using the YOLO framework, MoViNets for efficient video recognition, and TSM-MobileNetV2 for lightweight temporal modelling.

BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose, YOLOPose, and MoveNet were benchmarked under comparable conditions. Throughput and mean pose-inference time were recorded. The evaluation also considered whether each model could run on the robot and produce the COCO-17 keypoint representation required by the downstream feature-extraction and recognition pipeline.

MoveNet Lightning int8, executed through LiteRT, was selected because it provided the most suitable trade-off between speed, keypoint compatibility, and deployment feasibility.

2.3 Training, export, and inference

No new video recordings were collected for model training. Instead, the recognition models were trained using existing COCO-17 skeleton data published by OpenMMLab for NTU RGB+D 60 [4] (wave and clapping) and UCF101 [5] (pushup, squat, and jumping jacks). HMDB51 was used only as an external test set. All other dataset classes were labelled as none.

Each clip was resampled from 30 to approximately 10 FPS to match the robot camera. The clips were then divided into temporal windows using the same feature-extraction code as the live system: 1.5-second windows with nine features for the binary wave model, and 3-second windows with 34 features for the six-class action model. To approximate the table-mounted camera perspective, each clip was also windowed with leg and hip keypoints removed.

Random forests were trained using scikit-learn [6]. The wave model used 100 trees with a maximum depth of six, while the action model used 300 class-balanced trees. The models were evaluated using five-fold GroupKFold by subject or footage group, following [7]. Precision, recall, and F1 scores were calculated from out-of-fold predictions. A confidence threshold was tuned for each class using these predictions; windows below the threshold were classified as none.

The final forests were fitted on all training windows and exported to compressed `.npz` files containing the decision-tree node arrays. On the robot, a NumPy-only implementation traverses these trees and reproduces scikit-learn's `predict_proba` behaviour, removing the need for scikit-learn during inference.

2.4 Robot integration

The applications use the Reachy Mini Python SDK (`reachy_mini` 1.10.0, matched to the daemon version) for motion and the daemon REST API on port 8000 for state management.

Before camera, tracking, or movement operations, a standard-library preflight procedure checks network reachability, daemon and backend state, running applications and movements, motor mode, sleep pose, camera ownership, and tracker frames. When automatic correction is enabled, the procedure enables the motors, executes the wake-up movement, and reacquires the camera. It returns an explicit ready, not-ready, or unreachable status. A context manager combines preflight checks, SDK connection, and cleanup, including stopping head tracking and optionally returning the robot to its sleep pose.

Because the vision loop operates at approximately 10 FPS while the daemon controls the robot at 50 Hz, a dedicated thread streams `set_target` commands at 50 Hz. Each recognised action produces a distinct antenna movement pattern defined by amplitude, frequency, and a whole number of cycles ending at the neutral pose. Head and body following use an integrating controller based on the image-position error of the torso centre. Body yaw extends the tracking range beyond the head's 60-degree movement limit relative to the body.

Additional integrations include an annotated MJPEG camera stream for browser-based observation without a local display, a remote-inference mode that sends low-resolution frames (192×108 pixels) to a laptop GPU and receives keypoints over HTTP, and a coach workflow for training personalised models on the robot's CPU. In the coach workflow, scikit-learn runs in a low-priority subprocess to keep the camera and tracking loops responsive. A failed training run does not overwrite the previous model. Deployment uses an isolated virtual environment to avoid modifying the robot's system environment.

2.5 Testing and evaluation

Testing and evaluation examined the technical feasibility of the system against the requirements established during the research phase. Model performance was assessed using the benchmark results and the out-of-fold precision, recall, and F1 scores. HMDB51 provided an external test set beyond the datasets used for training. Camera-framing experiments considered the effect of removing hip and leg keypoints on the information available for activity recognition.

The evaluation also considered inference speed, CPU-only deployment, compatibility with the Reachy Mini's camera and movement interfaces, and system responsiveness during simultaneous recognition and tracking. The findings were used to identify technical limitations and formulate recommendations for further development. The project focused on technical feasibility; it did not validate a clinical assessment instrument or establish clinical suitability for physiotherapy.

3 Results

3.2 Pose-model performance

The benchmark compared BlazePose, MoViNet-A0, TSM-MobileNetV2, ViTPose and MoveNet as candidate components for the perception pipeline. MoveNet Lightning int8 achieved an average of 23.80 FPS, with a mean pose-inference time of approximately 34 ms on the Reachy Mini. This was the most suitable result for the project's CPU-only deployment target. The model also produces the COCO-17 keypoints expected by the temporal feature and recognition code.

The comparison also exposed deployment constraints. BlazePose and ViTPose are useful comparison models, but the robot cannot use the available BlazePose MediaPipe binary because it requires AES instructions unavailable on the Reachy's Cortex-A72; ViTPose's detector-and-pose pipeline is intended for the laptop or simulation environment. MoViNet-A0 and TSM-MobileNetV2 were considered for temporal classification rather than selected as replacements for the keypoint-based pipeline. Consequently, MoveNet int8 was used for robot inference, while the shared downstream representation remained model-independent.

3.3 Wave recognition

On two recorded sessions with our own robot (416 windows), the rule-based detector reached F1 0.811 (precision 1.000, recall 0.682). The NTU-trained forest, which never saw these recordings, reached F1 0.951 and outperformed a forest trained on the sessions themselves (F1 0.884, historical only). On unseen NTU subjects, where the negatives include deliberately confusable gestures (salute, selfie and brushing hair), F1 was 0.556. On HMDB51 movie clips it was 0.065, a deliberately pessimistic external check.

3.4 Six-class action recognition

Table 1: Out-of-fold F1 on held-out subjects or footage groups. Waist-up framing is simulated by removing the hip, knee and ankle keypoints.

Class	Full body	Waist-up
none	0.902	0.855
wave	0.918	0.873
squat	0.922	0.803
clapping	0.898	0.899
jumping_jacks	0.887	0.861
pushup	0.381	0.137

These are in-domain scores. A source confound inflates wave and clapping, both taken from the studio-recorded NTU corpus. On our robot recordings wave drops to 0.581, the figure to quote for a robot in a room, and clapping has not yet been measured on robot footage. Squats are almost never false positives (precision 0.998) but miss about one repetition in seven (one in three waist-up). Push-ups (19 training groups, per-fold F1 0.06–0.71) are insufficiently measured and are not demonstrated. Without crop augmentation, waist-up F1 for wave, squat and clapping fell to 0.000.

3.5 Engineering results

- One model file serves both simulation and robot. The six-class model is 5.3 MB and the wave model 0.25 MB, with no scikit-learn dependency at runtime.
- On the robot, the full wave pipeline runs at 8.7 FPS: 34 ms pose estimation and 3 ms motion processing (features and forest). The camera (approximately 10 FPS) is the remaining bottleneck.
- Antenna gestures are rendered smoothly at 50 Hz independently of the vision rate. Head and body following reached 75 degrees of gaze, compared with 35 degrees for the head alone.
- Preflight turned the robot's frequent post-boot state (asleep, motors off and camera released) into an automatic fix rather than a runtime failure.

4 Discussion

Maximum of 400 words.

The discussion describes a critical view (reflection) of the author(s) on the work delivered as a whole:

- Which aspects have contributed positively to the results achieved and why?
- Which aspects have been limiting for the results achieved and why?

Aspects can be considered broadly here. For example, process choices (method), design decisions (results) or other (whether or not externally determined) aspects that have influenced the results

5 Conclusions

Maximum of 400 words. The conclusion describes the extent to which the original goal of the assignment has been met with the help of the achieved (factual, substantive) results. In addition, any recommendations for follow-up work are described:

- Which artefacts have/have not been realised?
- To what extent have any boundary conditions/requirements been met?
- To what extent can the results be used/used for the original purpose?
- What recommendations are made on the basis of this, both towards practice and any necessary follow-up research into previously carried out work?

References

[1] A. R. Hevner, "A three cycle view of design science research," *Scandinavian Journal of Information Systems*, vol. 19, no. 2, pp. 87–92, 2007. [Online]. Available: https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1017&context=sjis

[2] GeeksforGeeks. "What is prototyping? definition, types, qualities and more," Accessed: Oct. 26, 2026. [Online]. Available: https://www.geeksforgeeks.org/software-engineering/what-is-a-prototype-and-how-to-create-it/

[3] Agile Scrum Group. "Agile vs waterval: Wanneer gebruik je welke methode?" Accessed: Oct. 26, 2026. [Online]. Available: https://agilescrumgroup.nl/agile-vs-waterval/

[4] A. Shahroudy, J. Liu, T.-T. Ng, and G. Wang, "Ntu rgb+d: A large scale dataset for 3d human activity analysis," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 1010–1019. DOI: 10.1109/CVPR.2016.115

[5] K. Soomro, A. R. Zamir, and M. Shah, "Ucf101: A dataset of 101 human actions classes from videos in the wild," Center for Research in Computer Vision, University of Central Florida, Tech. Rep. CRCV-TR-12-01, 2012. arXiv: 1212.0402.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011.

[7] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," *GigaScience*, vol. 6, no. 5, pp. 1–9, 2017. DOI: 10.1093/gigascience/gix019

Appendices

A Additional material